

UNIVERSIDAD NACIONAL DE RAFAELA

Ingeniería en Computación

Comunicación de Datos — 4° año — 1° Cuatrimestre 2026

Trabajo Práctico N° 2

SISTEMA DE CONTROL INALÁMBRICO INFRARROJO BIDIRECCIONAL

*Sincronización de Estado Cromático entre Nodos Maestro–Esclavo mediante
Protocolo NEC*

Autores

Bevilacqua, Francisco | Clement, Sebastián

Docentes:

Mg. Ing. Martín Pico | Ing. Milton Pozzo

Fecha de entrega:

31 de mayo de 2026

1. Introducción

La comunicación por infrarrojos (IR) constituye uno de los estándares más difundidos en sistemas embebidos y control hogareño, gracias a su bajo costo, reducido consumo energético y simplicidad de implementación física. Sin embargo, detrás de la aparente sencillez de un control remoto subyace uno de los desafíos más críticos de la ingeniería de redes: la sincronización de estado entre nodos.

El presente trabajo práctico propone diseñar y desarrollar un sistema de control inalámbrico infrarrojo bidireccional que permita la sincronización de estado cromático entre dos nodos Arduino UNO, emulando la lógica de control de set-point de un sistema de climatización moderno. En lugar de temperatura, el parámetro de estado es un valor de color RGB representado por un LED RGB.

1.1 Analogía con sistemas de climatización

Un aire acondicionado moderno ilustra el problema con precisión: el control remoto (Maestro) exhibe en pantalla un set-point de temperatura que debe coincidir exactamente con el valor procesado por la unidad interna (Esclavo). Si una obstrucción momentánea del haz infrarrojo impide la recepción del comando, se produce una incoherencia de estado: el usuario cree haber fijado 24 °C, pero la unidad continúa operando a 22 °C. Para mitigar este escenario, los sistemas avanzados implementan protocolos de handshaking y retransmisión, donde el Esclavo debe confirmar la recepción para que el Maestro actualice su registro local.

En este trabajo, los LEDs RGB actúan como el elemento de estado visible. El desafío técnico es idéntico: garantizar que dos dispositivos físicamente separados compartan la misma información cromática en todo momento, con resiliencia ante pérdidas de trama, cortes de energía e interferencia ambiental.

1.2 Conceptos implementados

- **Protocolización:** empaquetado de los valores RGB más una cabecera de identificación en una única trama de 32 bits transmitida en un solo frame NEC.
- **Control de errores y flujo:** verificación de trama mediante cabecera fija (0xAA / 0xBB) y filtro de tramas de repetición NEC.
- **Handshaking bidireccional completo por IR:** el Esclavo responde con un ACK infrarrojo portando los mismos valores RGB recibidos, permitiendo al Maestro validar la integridad de la operación byte a byte.
- **Persistencia de estado en EEPROM:** almacenamiento del último color confirmado en memoria no volátil para recuperación automática tras ciclos de energización.
- **ARQ stop-and-wait:** hasta 3 reintentos automáticos del Maestro si no recibe confirmación dentro de un timeout de 2 segundos.
- **Idempotencia del receptor:** el Esclavo puede procesar la misma trama múltiples veces sin generar inconsistencias de estado.

2. Marco Teórico

2.1 Comunicación inalámbrica por infrarrojos

La comunicación infrarroja es una forma de transmisión inalámbrica óptica que utiliza radiación electromagnética en el espectro cercano al infrarrojo, con longitudes de onda típicas entre 850 nm y 950 nm. En los sistemas de control remoto de consumo masivo, la señal útil es modulada sobre una portadora en el rango de 30 kHz a 56 kHz. Esta modulación cumple una función esencial: permite que el receptor discrimine la señal del transmisor del ruido lumínico ambiental (luz solar, lámparas fluorescentes, etc.), que se manifiesta como una componente de continua (DC) o de frecuencias muy bajas respecto a la portadora.

El esquema de modulación empleado es OOK (On-Off Keying) sobre portadora: el LED emisor enciende y apaga a la frecuencia de la portadora durante los períodos de «burst» (nivel activo), y permanece apagado durante los «espacios» (nivel inactivo). El receptor TSOP demodula esta señal y entrega al microcontrolador una señal lógica en nivel invertido: nivel alto en reposo, nivel bajo durante burst.

Dada la naturaleza óptica del canal, la comunicación IR es inherentemente half-duplex sobre el medio compartido: si ambos dispositivos emiten simultáneamente, sus haces se superponen en el receptor y la señal resultante es indecodificable. Esto impone la necesidad de gestionar el turno de transmisión por software, lo cual se aborda en detalle en la sección de implementación.

2.2 Protocolo NEC

2.2.1 Estructura general de la trama

El protocolo NEC es uno de los estándares de comunicación infrarroja más difundidos en la industria de electrodomésticos. Fue desarrollado por NEC Corporation y opera a una frecuencia portadora de 38 kHz, empleando una codificación de tipo Pulse Distance Modulation (PDM), donde la información se codifica en la distancia temporal entre pulsos de igual duración, y no en la duración del pulso en sí.

La trama NEC estándar está compuesta por los siguientes elementos temporales:

- **Pulso de inicio (AGC burst):** 9 ms de señal activa (burst) seguido de 4,5 ms de espacio. Este pulso permite al receptor ajustar su ganancia automática (AGC) y sincronizarse con el inicio de la trama.
- **32 bits de datos:** cada bit se codifica con 562,5 μ s de burst seguido de un espacio cuya duración determina el valor lógico.
- **Bit '0':** 562,5 μ s burst + 562,5 μ s espacio $\rightarrow$ duración total $\sim 1,125$ ms.
Bit '1': 562,5 μ s burst + 1.687,5 μ s espacio $\rightarrow$ duración total $\sim 2,25$ ms.
- **Pulso de parada:** 562,5 μ s de burst final para marcar el fin de la trama.
- **Código de repetición:** si el emisor mantiene el botón presionado, envía cada ~ 108 ms un pulso de 9 ms burst + 2,25 ms espacio + 562,5 μ s burst, indicando que el último comando debe repetirse.

En el protocolo NEC original, los 32 bits se organizan como: [Dirección 8 bits] + [$\sim$ Dirección 8 bits] + [Comando 8 bits] + [$\sim$ Comando 8 bits], donde « $\sim$ » denota el complemento lógico, utilizado como verificación de integridad básica. En este proyecto, esta estructura original fue reemplazada por un formato propio de 32 bits que transporta [Cabecera 8 bits] + [R 8 bits] + [G 8 bits] + [B 8 bits], aprovechando la totalidad del campo de datos del protocolo para transportar información útil.

2.2.2 Uso de sendNECRaw() y limitación de sendNEC()

La librería IRremote v4.x provee la función sendNEC(address, command, repeats), que sigue la especificación original del protocolo: espera una dirección de 16 bits y un comando de 8 bits, y agrega automáticamente los bytes invertidos. Al pasarle una variable de 32 bits (uint32_t) como parámetro «command», la función la trunca a los 8 bits menos significativos, descartando los bytes de los canales G, B y la cabecera.

Solución adoptada — sendNECRaw(): la función sendNECRaw(uint32_t data, uint8_t repeats) transmite los 32 bits como payload completo sin modificación, interpretación ni adición de campos invertidos. El receptor obtiene estos 32 bits íntegros mediante IrReceiver.decodedIRData.decodedRawData. Esta función permite usar el frame NEC como transporte de datos arbitrarios de 32 bits, que es exactamente el requerimiento de este sistema.

2.2.3 Definición de IR_SEND_PIN antes del include

Un requerimiento crítico de la librería IRremote v4.x es que la macro IR_SEND_PIN debe definirse antes del #include <IRremote.hpp>. Esta macro configura en tiempo de compilación el pin físico del emisor, vinculándolo al hardware de timer correspondiente del microcontrolador. Si se define después del #include, la librería ya ha configurado su pin por defecto y la definición tardía no tiene efecto. En ambos firmwares, esta convención se respetó explícitamente como primera directiva del archivo.

2.3 Comparativa de protocolos IR evaluados

Criterio	NEC	RC-5 (Philips)	Sony SIRC
Frecuencia portadora	38 kHz	36 kHz	40 kHz
Bits de datos útiles	32 bits	14 bits	12–20 bits
Codificación	Pulse Distance (PDM)	Biphase (Manchester)	Pulse Width (PWM)
Duración trama	~67,5 ms	~24,9 ms	~45 ms
Verificación nativa	Bytes invertidos	Ninguna	Ninguna
Repetición	Código especial	Bit de toggle	Reenvío 3×
Soporte sendNECRaw()	Sí (nativo)	No aplicable	No aplicable
Capacidad [H R G B]	Suficiente (32 bits)	Insuficiente (14 bits)	Insuficiente
Módulo receptor KY-022	Compatible (38 kHz)	Incompatible (36 kHz)	Incompatible (40 kHz)

Justificación de la elección de NEC: el protocolo NEC fue seleccionado porque:

- (a) sus 32 bits de payload son suficientes para transportar el byte de cabecera más los tres canales RGB en una única trama atómica, sin fragmentación;
- (b) opera exactamente a 38 kHz, frecuencia nativa del receptor KY-022 (TSOP38238);
- (c) es el protocolo con mayor soporte y documentación en la librería IRremote;
- (d) su codificación PDM es más robusta ante variaciones de amplitud de señal que la codificación PWM de Sony SIRC.

2.4 ARQ Stop-and-Wait

ARQ (Automatic Repeat reQuest) es un mecanismo de la capa de enlace que combina detección de errores con retransmisión automática. En la variante Stop-and-Wait, el emisor transmite una única unidad de datos y aguarda la confirmación (ACK) antes de enviar la siguiente. Si el ACK no llega dentro de un tiempo límite (timeout), retransmite la misma unidad. Este sistema implementa ARQ stop-and-wait con timeout de 2.000 ms y hasta 3 reintentos (MAX_REINTENTOS = 3).

2.5 Idempotencia del receptor

Idempotencia es la propiedad de una operación cuyo resultado es el mismo independientemente de cuántas veces se ejecute. El Esclavo implementa esta propiedad: procesar la misma trama RGB válida N veces produce invariablemente el mismo color en el LED y el mismo estado en la EEPROM. Esta propiedad es fundamental para el correcto funcionamiento del ARQ: cuando el Maestro reenvía una trama porque no recibió ACK, el Esclavo puede procesarla nuevamente sin inconsistencias ni efectos secundarios.

2.6 Memoria EEPROM en AVR

El microcontrolador ATmega328P (Arduino UNO) dispone de 1 KB de memoria EEPROM (Electrically Erasable Programmable Read-Only Memory) no volátil, con una vida útil mínima garantizada de 100.000 ciclos de escritura por celda. La librería EEPROM.h del core AVR provee las funciones read(), write() y update(). La estrategia de escritura es crítica: cada ciclo de escritura consume uno de los 100.000 disponibles. En el Esclavo, la EEPROM se escribe en cada comando recibido; en el Maestro, solo al confirmar un cambio exitoso, reduciendo la frecuencia de escritura.

2.7 Canal Half-Duplex sobre medio compartido IR

La comunicación IR es intrínsecamente half-duplex sobre el espacio físico: dos dispositivos no pueden emitir simultáneamente en el mismo entorno sin que sus señales colisionen en el receptor. La gestión del turno de transmisión se implementa por software en ambos firmwares mediante la secuencia IrReceiver.stop() → IrSender.sendNECRaw() → IrReceiver.start(). Esta secuencia garantiza que el receptor propio queda inhabilitado durante la transmisión propia, evitando que el dispositivo capture su propio eco.

3. Diseño del Sistema

3.1 Arquitectura general

El sistema está compuesto por dos nodos Arduino UNO que se comunican de forma bidireccional mediante un enlace infrarrojo inalámbrico. A diferencia de una primera iteración del diseño que empleaba un cable digital dedicado para el ACK, la versión final implementa el canal de retorno íntegramente por IR, dotando a cada nodo de un módulo emisor KY-005 y un módulo receptor KY-022. Ambos nodos comparten el mismo protocolo de trama, la misma lógica de validación por cabecera y el mismo esquema de persistencia en EEPROM.

3.2 Evolución del diseño: del ACK por cable al ACK bidireccional por IR

En la primera versión del sistema (pre-informe), el canal de retorno (ACK) se implementó mediante una línea digital directa entre los dos Arduinos (pin 8 ↔ pin 8 con GND común). Esta solución era funcional, pero presentaba dos limitaciones relevantes:

- (a) requería un cable físico entre los dispositivos, eliminando la ventaja de la comunicación inalámbrica;
- (b) el ACK era una simple señal binaria (HIGH/LOW), sin capacidad de transportar información de estado.

La versión final elimina completamente el cable de ACK y lo reemplaza por una trama IR de 32 bits con cabecera 0xBB que transporta los mismos valores RGB recibidos. Esto aporta una ventaja fundamental: el Maestro puede comparar byte a byte los valores RGB del ACK contra los que envió, verificando no solo que el Esclavo recibió “algo”, sino que recibió exactamente el color correcto. Adicionalmente, el sistema es puramente inalámbrico.

Para implementar el ACK por IR, cada nodo fue equipado con un par completo de módulos (emisor + receptor), seleccionando pines distintos para TX y RX de manera que no colisionen con los recursos de hardware del ATmega328P (timers, interrupciones).

3.3 Estructura de la trama de datos

Se diseñó una trama de 32 bits que aprovecha la totalidad del payload NEC:

Bits	Campo	Tamaño	Valor / Rango	Función
[31:24]	Cabecera	8 bits	0xAA (CMD) / 0xBB (ACK)	Identifica el tipo de trama y actúa como primer filtro de validez
[23:16]	Red (R)	8 bits	0–255	Componente rojo del color RGB
[15: 8]	Green (G)	8 bits	0–255	Componente verde del color RGB
[7: 0]	Blue (B)	8 bits	0–255	Componente azul del color RGB

Justificación del valor 0xAA como cabecera: el patrón binario 10101010 presenta alternancia de bits, lo que asegura que la trama nunca comienza con bits cero en el byte

más significativo. Esto es importante porque algunas implementaciones de `sendNECRaw()` pueden comportarse de manera inesperada con tramas que comienzan con ceros en los bits más significativos. Adicionalmente, la alternancia de bits le otorga a la cabecera una firma espectral reconocible en el osciloscopio, facilitando la identificación visual de inicio de trama.

Justificación del valor 0xBB como cabecera de ACK: el patrón 10111011 es distinto de 0xAA, lo que permite al Maestro distinguir inequívocamente un ACK de un comando. Si el Maestro captara accidentalmente el eco de su propio comando durante la ventana de espera del ACK, la verificación de cabecera (`cab == 0xBB`) lo descartaría, evitando falsos positivos.

Ejemplo de construcción para el color Amarillo (R=255, G=200, B=0):

Campo	Valor decimal	Valor hex	Binario
Cabecera CMD	170	0xAA	1010 1010
R	255	0xFF	1111 1111
G	200	0xC8	1100 1000
B	0	0x00	0000 0000
Trama 32 bits completa	—	0xAAFFC800	10101010 11111111 11001000 00000000

3.4 Secuencia de colores

Índice	Color	R	G	B	Trama CMD
0	Rojo	255	0	0	0xAA000000
1	Amarillo	255	200	0	0xAAFFC800
2	Verde	0	255	0	0xAA00FF00
3	Celeste	0	255	255	0xAA00FFFF
4	Azul	0	0	255	0xAA0000FF
5	Lila	180	0	255	0xAAB400FF
6	Blanco	255	255	255	0xAFFFFFFF
7	Rosa	255	105	180	0xA AFF69B4

3.5 Asignación de pines de hardware

3.5.1 Dispositivo 1 — Esclavo

Componente	Pin módulo	Pin Arduino	Recurso AVR	Justificación
KY-022 (Receptor IR)	S (DATA)	D2	INT0	IRremote requiere INT0 para captura de trama por interrupción externa

Componente	Pin módulo	Pin Arduino	Recurso AVR	Justificación
KY-005 (Emisor IR)	S (DATA)	D3	OC2B / Timer2	IR_SEND_PIN=3; Timer2 genera la portadora de 38 kHz por hardware
LED RGB — Canal R	Ánodo	D9	OC1A / Timer1	Señal PWM de 8 bits; Timer1 no colisiona con Timer2 del IR
LED RGB — Canal G	Ánodo	D10	OC1B / Timer1	Ídem; Timer1
LED RGB — Canal B	Ánodo	D6	OC0A / Timer0	Timer0 no colisiona con Timer2 del IR TX
KY-022 VCC / GND	V+ / GND	5V / GND	—	Alimentación módulo TSOP38238
KY-005 VCC / GND	V+ / GND	5V / GND	—	Alimentación LED IR KY-005
LED RGB — Cátodo	—	GND	—	LED cátodo común; referencia de tierra PWM

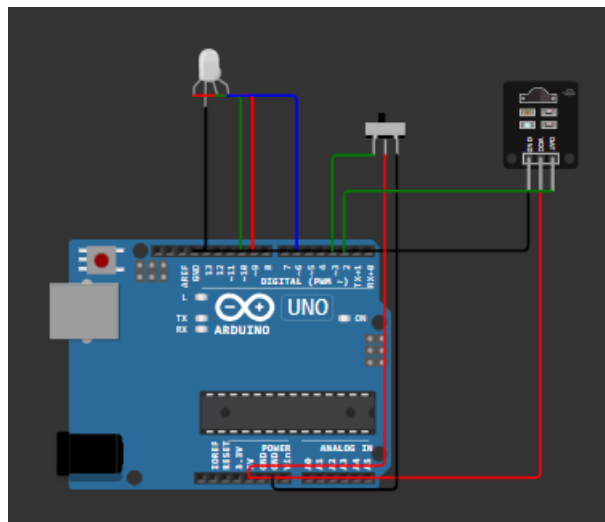


Figura: Esquema eléctrico — Dispositivo 1 (Esclavo)

3.5.2 Dispositivo 2 — Maestro

El Maestro requirió una asignación de pines distinta a la del Esclavo, dado que el emisor IR debió alejarse de los pines controlados por Timer2 para evitar interferencias con el receptor:

Componente	Pin módulo	Pin Arduino	Recurso AVR	Justificación
KY-005 (Emisor IR)	S (DATA)	D4	GPIO (Timer libre)	IR_SEND_PIN=4; en IRremote v4, el pin TX puede ser cualquier GPIO cuando se usa sendNECRaw(); Timer2 queda libre
KY-022 (Receptor IR)	S (DATA)	D2	INT0	INT0 para captura del ACK del Esclavo

Componente	Pin módulo	Pin Arduino	Recurso AVR	Justificación
Pulsador	Terminal 1	D7	GPIO	INPUT_PULLUP interno; activo en LOW al presionar
Pulsador	Terminal 2	GND	—	Referencia de tierra del pulsador
LED RGB — Canal R	Ánodo	D9	OC1A / Timer1	PWM de 8 bits; Timer1
LED RGB — Canal G	Ánodo	D10	OC1B / Timer1	PWM de 8 bits; Timer1
LED RGB — Canal B	Ánodo	D6	OC0A / Timer0	PWM de 8 bits; Timer0
LED RGB — Cátodo	—	GND	—	LED cátodo común

Nota sobre el pin TX del Maestro (D4): en la versión final del firmware del Maestro, IR_SEND_PIN se define en D4 en lugar de D3. Esta decisión se tomó para evitar colisiones con el Timer2 del ATmega328P, recurso compartido entre el emisor IR (que lo usa para la portadora de 38 kHz) y otros periféricos. Al mover el pin TX a D4, el Timer2 queda exclusivamente disponible para la generación de la portadora. En el Esclavo, el pin TX permanece en D3 (OC2B), el pin canónico de Timer2, dado que en ese nodo la carga de timers es diferente.

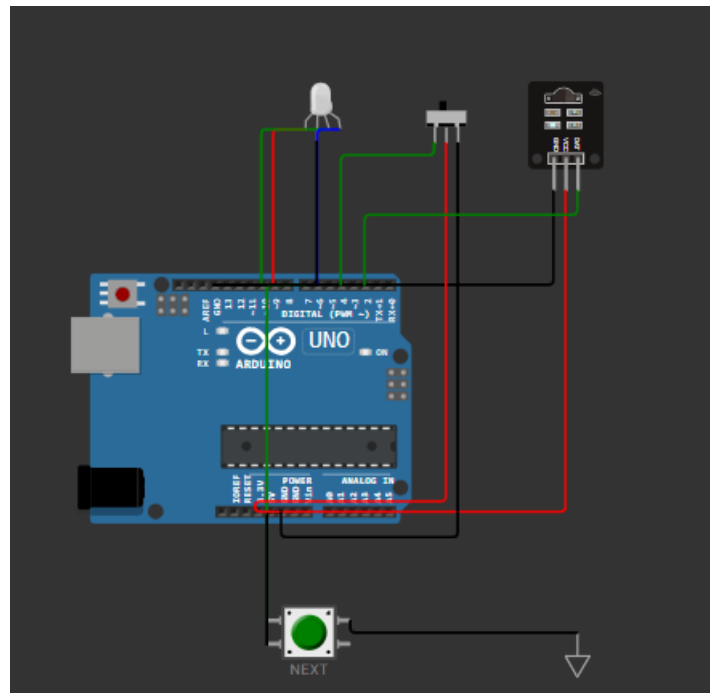


Figura: Esquema eléctrico — Dispositivo 2 (Maestro)

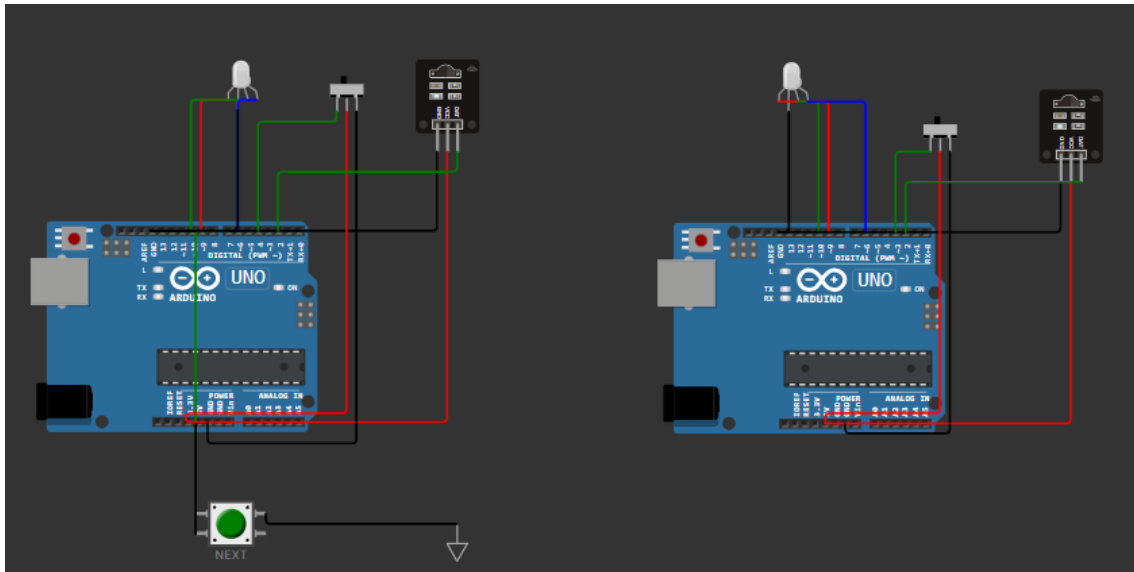


Figura: Simulación en Wokwi — Sistema completo Maestro-Eslavo

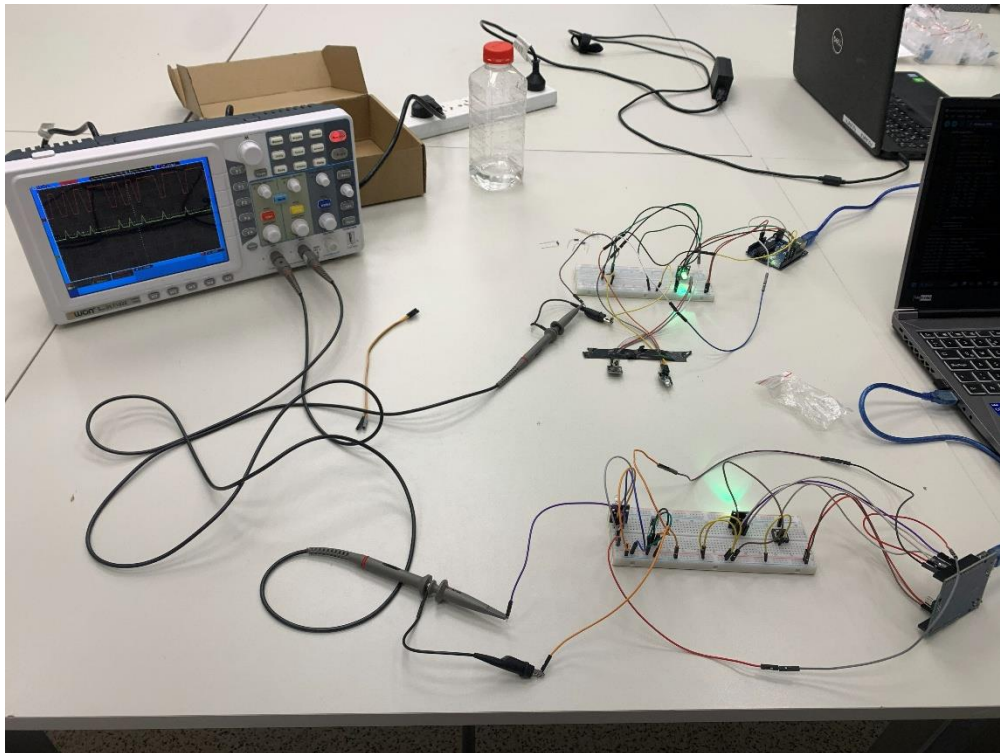


Figura: Sistema completo – Análisis de señal en osciloscopio del color VERDE.

4. Implementación del Firmware

4.1 Librería IRremote v4.x

Se utilizó la librería IRremote v4.7.1 de Armin Joachimsmeier (repositorio Arduino-IRremote en GitHub), instalada desde el Administrador de Librerías del Arduino IDE 2.3.8.

Función / Campo	Parámetros	Tipo retorno	Descripción
IrReceiver.begin(pin, feedback)	uint8_t pin, bool feedback	void	Inicia el receptor en el pin indicado. DISABLE_LED_FEEDBACK evita parpadeo del pin 13 durante recepción
IrReceiver.decode()	—	bool	Retorna true cuando hay una trama completamente decodificada en el buffer
IrReceiver.decodedIRData.decodedRawData	—	uint32_t	Los 32 bits del payload NEC recibido, sin inversión ni procesamiento adicional
IrReceiver.decodedIRData.flags	—	uint8_t	Flags de estado; IRDATA_FLAGS_IS_REPEAT indica trama de repetición automática NEC
IrReceiver.resume()	—	void	Libera el buffer del receptor para poder recibir la siguiente trama
IrReceiver.stop()	—	void	Pausa la interrupción del receptor; impide capturar el eco del emisor propio
IrReceiver.start()	—	void	Reactiva el receptor tras una transmisión
IrSender.sendNECRaw(data, repeats)	uint32_t data, uint8_t repeats	void	Transmite 32 bits como payload NEC íntegro. Solución a la limitación de sendNEC() que trunca a 8 bits

4.2 Librería EEPROM.h (built-in AVR Core)

Función	Parámetros	Tipo retorno	Descripción
EEPROM.read(address)	int address	uint8_t	Lee 1 byte de la dirección especificada
EEPROM.write(address, value)	int, uint8_t	void	Escribe 1 byte. Consume 1 ciclo de escritura siempre, independientemente del valor previo

Función	Parámetros	Tipo retorno	Descripción
EEPROM.update(address, value)	int, uint8_t	void	Escribe solo si el byte actual difiere del valor nuevo. Recomendado para preservar ciclos de vida de la celda

4.3 Dispositivo 1 — Esclavo: flujo de ejecución

4.3.1 Inicialización

Al energizarse, el Esclavo ejecuta la siguiente secuencia:

1. Inicializa el puerto serie a 9.600 bps para logging/debugging.
2. Configura los pines D9, D10, D6 como OUTPUT para los canales R, G, B del LED RGB.
3. **Inicia IrReceiver en D2** con DISABLE_LED_FEEDBACK: el parámetro false evita que la librería toggle el pin 13 (LED integrado de la placa) durante la recepción, lo que podría generar ruido eléctrico o confusión visual.
4. **Lee la EEPROM:** si el byte en la dirección EEPROM_VALID (addr 0) es igual al magic byte 0xA5, los datos son válidos → lee R, G, B de las direcciones 1, 2 y 3 y aplica el color recuperado al LED. Si no es 0xA5, la EEPROM está virgen o corrompida → aplica negro/apagado (0,0,0) como estado seguro.
5. Entra en el loop principal en modo escucha activa.

4.3.2 Loop principal y recepción de tramas

En cada iteración del loop(), IrReceiver.decode() retorna true cuando hay una trama completa en el buffer. El primer paso es filtrar las tramas de repetición NEC: cuando el emisor mantiene la señal por más de ~108 ms, la librería genera automáticamente tramas de repetición marcadas con el flag IRDATA_FLAGS_IS_REPEAT. Estas deben descartarse llamando a IrReceiver.resume() y retornando sin procesar, ya que el sistema no implementa lógica de repetición de comandos.

El segundo paso es leer decodedIRData.decodedRawData (los 32 bits del payload) antes de llamar a IrReceiver.resume(). Esta secuencia es crítica: resume() libera el buffer interno, sobrescribiendo los datos. Si se llama antes de leer, el dato se pierde.

Una vez leída la trama, el Esclavo desempaqueta los 4 bytes mediante desplazamientos de bits y verificación de la cabecera:

- **Cabecera = bits [31:24], obtenidos con (trama >> 24) & 0xFF.**
- **R = bits [23:16], obtenidos con (trama >> 16) & 0xFF.**
- **G = bits [15:8], obtenidos con (trama >> 8) & 0xFF.**
- **B = bits [7:0], obtenidos con trama & 0xFF.**

Si cabecera == 0xAA (trama de comando válida): (1) aplica el color al LED; (2) guarda R, G, B en EEPROM junto al magic byte; (3) pausa IrReceiver con stop(); (4) espera 100 ms para que el Maestro reactive su receptor; (5) construye y envía el ACK con cabecera 0xBB y los mismos R, G, B; (6) reactiva IrReceiver con start().

Si cabecera ≠ 0xAA (trama desconocida o ruido ambiental): descarta silenciosamente sin modificar estado ni enviar ACK.

4.3.3 Gestión half-duplex en el Esclavo

La secuencia `IrReceiver.stop()` → `delay(100)` → `IrSender.sendNECRaw(ack, 0)` → `IrReceiver.start()` implementa la gestión del canal half-duplex:

- **stop()**: deshabilita la interrupción INT0 del receptor, impidiendo que el KY-022 del Esclavo capture la señal de su propio KY-005 durante la transmisión del ACK.
- **delay(100)**: proporciona un margen temporal de 100 ms para que el Maestro haya finalizado su transmisión y activado su propio receptor antes de que el ACK llegue. Este delay es el parámetro más crítico del timing del sistema: si es demasiado corto, el Maestro aún no está en modo escucha; si es demasiado largo, podría agotarse el timeout del Maestro.
- **sendNECRaw(ack, 0)**: transmite el ACK de 32 bits. El parámetro 0 indica cero repeticiones automáticas: el ACK se envía exactamente una vez.
- **start()**: reactiva el receptor para seguir escuchando nuevos comandos.

4.4 Dispositivo 2 — Maestro: flujo de ejecución

4.4.1 Inicialización

6. Inicializa el puerto serie a 9.600 bps.
7. **Configura el pulsador en D7 como INPUT_PULLUP**: la resistencia de pull-up interna del ATmega328P (~50 kΩ) mantiene el pin en HIGH en reposo. Al presionar, el pin conecta a GND → nivel LOW. Esto elimina la necesidad de una resistencia externa.
8. Configura los pines D9, D10, D6 como OUTPUT para el LED RGB.
9. Inicia IrReceiver en D2 con DISABLE_LED_FEEDBACK.
10. **Lee la EEPROM**: si el magic byte es válido, recupera el índice del último color exitoso (addr 1). Un sanity check verifica que el índice esté en el rango [0, 7]; si no, lo resetea a 0. Esto protege ante escrituras parciales de EEPROM por corte de energía.
11. Aplica el color del índice recuperado al LED local.

4.4.2 Loop: detección de pulsador con debounce

El `loop()` implementa detección de pulsador con antirrebote por software de doble lectura con delay de 50 ms: la primera lectura detecta el flanco descendente; el delay absorbe los rebotes mecánicos del pulsador (típicamente 10–50 ms en pulsadores estándar); la segunda lectura confirma que la señal sigue en LOW, descartando glitches transitorios. Este método es suficiente y no requiere hardware adicional.

Detectada una pulsación válida, el Maestro calcula el siguiente índice con módulo 8: $(\text{colorActual} + 1) \% 8$, y lee los valores R, G, B correspondientes de la tabla `COLORES[]`.

4.4.3 Envío con handshaking (enviarColor())

La función `enviarColor(r, g, b)` implementa el ciclo completo de envío y verificación:

12. **Construcción de trama**: empaqueta cabecera 0xAA, R, G, B en un `uint32_t` usando desplazamientos. El cast a `uint32_t` antes del desplazamiento es obligatorio: en la arquitectura AVR de 16 bits, `int` tiene 16 bits, y desplazar sin cast produciría overflow.
13. **Gestión half-duplex**: `IrReceiver.stop()` → `sendNECRaw(trama, 0)` → `IrReceiver.start()`. La misma secuencia que en el Esclavo, aplicada al rol inverso.

14. **Ventana de espera del ACK:** un bucle while espera hasta 2.000 ms. En cada iteración, si `IrReceiver.decode()` retorna true: filtra repeticiones NEC, lee la trama de ACK, verifica `cab == 0xBB` y que los bytes R, G, B del ACK coincidan exactamente con los enviados. Si coinciden → retorna true. Si el campo de 2 segundos se agota sin ACK válido → retorna false.

Importancia de la comparación byte a byte del ACK: esta verificación garantiza que el Esclavo aplicó exactamente el color solicitado y no un color residual de otro ciclo o una trama con un solo byte correcto. Un ACK con cabecera 0xBB pero valores RGB diferentes sería descartado como ACK inválido, desencadenando un reintento.

4.4.4 Lógica de reintentos ARQ

El loop de envío itera hasta `MAX_REINTENTOS = 3` veces. La condición del for (!ack) cortocircuita el bucle en el primer éxito (lazy evaluation). Entre reintentos se aplica un delay de 300 ms para permitir que el canal IR se despeje. Si los tres intentos fallan, `colorActual` no avanza y el LED del Maestro permanece en su color anterior, preservando la coherencia con el estado del Esclavo.

4.5 Diagrama de secuencia: ciclo nominal de comunicación

Paso	Maestro	Canal IR	Esclavo
1	Usuario presiona botón	—	En escucha (IrReceiver activo)
2	Debounce 50 ms, confirma pulsación	—	—
3	Calcula siguiente color $(idx+1)\%8$	—	—
4	<code>IrReceiver.stop()</code>	—	—
5	<code>sendNECRaw([0xAA R G B], 0)</code>	→ Trama CMD 32 bits a 38 kHz →	—
6	<code>IrReceiver.start()</code>	—	<code>IrReceiver.decode()</code> retorna true
7	Espera ACK (timeout 2000 ms)	—	Filtra <code>IRDATA_FLAGS_IS_REPEAT</code>
8	—	—	Lee trama, verifica <code>cab == 0xAA</code>
9	—	—	<code>setColor(R,G,B)</code> → aplica color al LED
10	—	—	<code>guardarEEPROM(R,G,B)</code>
11	—	—	<code>IrReceiver.stop()</code>
12	—	—	<code>delay(100 ms)</code>
13	—	← Trama ACK <code>[0xBB R G B]</code> ←	<code>sendNECRaw([0xBB R G B], 0)</code>
14	—	—	<code>IrReceiver.start()</code>
15	<code>IrReceiver.decode()</code> → lee ACK	—	—
16	Verifica <code>cab==0xBB</code> y R,G,B coinciden	—	—

Paso	Maestro	Canal IR	Esclavo
17	colorActual = siguiente; setColor(); guardarEEPROM()	—	—
18	delay(500 ms); espera botón liberado	—	En escucha

4.6 Mapa de EEPROM

4.6.1 Esclavo

Dirección	Nombre	Valor / Tipo	Descripción
0x00	EEPROM_VALID	0xA5 (magic byte)	Indica que los datos en las direcciones 1-3 son válidos. 0xFF = EEPROM virgen
0x01	EEPROM_R	uint8_t (0–255)	Último valor de canal Rojo confirmado y aplicado
0x02	EEPROM_G	uint8_t (0–255)	Último valor de canal Verde confirmado y aplicado
0x03	EEPROM_B	uint8_t (0–255)	Último valor de canal Azul confirmado y aplicado

4.6.2 Maestro

Dirección	Nombre	Valor / Tipo	Descripción
0x00	EEPROM_VALID	0xA5 (magic byte)	Indica que el índice en la dirección 1 es válido
0x01	EEPROM_INDEX	uint8_t (0–7)	Índice del último color exitosamente confirmado en COLORES[]

Estrategia de escritura del Maestro: el Maestro persiste solo el índice (1 byte) en lugar de los tres valores RGB, dado que puede reconstruir el color completo consultando la tabla COLORES[índice]. Esto reduce la escritura EEPROM por evento exitoso a 2 bytes (magic + índice) en lugar de 4 bytes, extendiendo la vida útil de las celdas.

5. Escenarios de Falla y Comportamiento del Sistema

El sistema fue diseñado para cubrir los escenarios de falla más relevantes de un enlace inalámbrico con confirmación. A continuación, se detalla cada uno.

5.1 Pérdida de energía del Esclavo

Situación: el Esclavo se desconecta durante operación normal. Al reconectarse, lee la EEPROM: si el magic byte 0xA5 está presente, recupera los últimos valores R, G, B y los aplica al LED. Si la EEPROM está virgen, aplica negro/apagado (estado seguro). El sistema no requiere intervención del Maestro para que el Esclavo recupere su estado.

5.2 Pérdida de energía del Maestro

Situación: el Maestro se desconecta. Al reconectarse, lee la EEPROM y recupera el índice del último color exitoso, aplicándolo al LED. El Esclavo mantiene su estado vigente en RAM y EEPROM. Al presionar el botón, el Maestro envía el siguiente color de la secuencia desde el índice recuperado, garantizando continuidad.

5.3 Trama corrupta (cabecera inválida)

Situación: interferencia IR ambiental u otro control remoto genera una trama con cabecera diferente de 0xAA. El Esclavo verifica la cabecera y descarta la trama sin modificar su estado ni enviar ACK. El Maestro detecta timeout (sin ACK en 2 segundos) y reintenta hasta 3 veces. Si persiste, el Maestro mantiene su color actual.

5.4 ACK perdido o corrupto — Estado Inconsistente Temporal

Situación: el Esclavo recibió el comando correctamente, aplicó el color y envió el ACK, pero este llegó corrupto o fue bloqueado. El Maestro no avanza su estado. Internamente: Esclavo en Color B, Maestro en Color A. El Maestro reintenta el mismo comando (ARQ). El Esclavo, siendo idempotente, puede procesar el mismo comando nuevamente y responder con un nuevo ACK. Si algún reintento prospera, el sistema converge. Si los 3 reintentos fallan, el Maestro permanece en Color A y el usuario puede presionar el botón nuevamente.

5.5 Recepción de trama duplicada (ARQ idempotente)

Situación: el Maestro reenvía la misma trama porque no recibió ACK. El Esclavo procesa la trama duplicada, aplica el mismo color (no hay cambio visible) y responde con el mismo ACK. EEPROM.write() escribe los mismos valores, sin efecto práctico. El resultado final es idéntico al de una primera recepción exitosa. La idempotencia garantiza que el reenvío no genera inconsistencias.

5.6 EEPROM virgen o con datos inválidos

Situación: primer uso del sistema o escritura incompleta por corte de energía durante un ciclo de escritura. El magic byte no es 0xA5 (valor por defecto de EEPROM virgen: 0xFF). Ambos dispositivos detectan la ausencia del magic byte y aplican sus estados por defecto seguros: Esclavo → negro/apagado (0,0,0); Maestro → índice 0 (Rojo). El sistema arranca en un estado conocido y definido.

5.7 Resumen

Escenario	Mecanismo de resolución	Estado final del sistema
Esclavo pierde energía	EEPROM + recuperación automática al arranque	Coherente
Maestro pierde energía	EEPROM + recuperación automática al arranque	Coherente
Trama corrupta (cab $\neq$ 0xAA)	Descarte silencioso + ARQ (hasta 3 reintentos)	Coherente si algún reintento prospera
ACK perdido o corrupto	ARQ (hasta 3 reintentos)	Coherente si algún reintento prospera
Trama duplicada por ARQ	Idempotencia del Esclavo	Coherente
EEPROM virgen o inválida	Bandera mágica + estado por defecto	Coherente

6. Análisis de Señales — Capa Física y de Enlace

Los análisis de señal se realizaron con un osciloscopio, conectando las sondas al pin de emisión D4 del Maestro y al pin de salida del receptor KY-022 (D2) del esclavo.

Canal ROJO: Salida del receptor IR hacia el dispositivo esclavo.

Canal AMARILLO: Salida del microcontrolador Maestro hacia el emisor IR.

6.1 Portadora de 38 kHz — Pin TX del emisor

Con la sonda conectada al pin TX durante una transmisión activa, se observa la portadora de 38 kHz generada por hardware mediante el Timer del Atmega328P durante los períodos de burst. En los períodos de espacio (entre bits), el pin permanece en nivel bajo. Canal AMARILLO.



Figura: Osciloscopio — Portadora IR 38 kHz en pin TX durante burst

6.2 Trama NEC completa — Pin RX del receptor KY-022

La salida del TSOP38238 (pin S del KY-022) entrega la señal demodulada e invertida: nivel alto en reposo, nivel bajo durante burst. Se observa el tren de pulsos NEC completo: el pulso de inicio de 9 ms, seguido de los 32 bits de datos con distintas distancias inter-pulso según el valor lógico de cada bit. Canal ROJO.

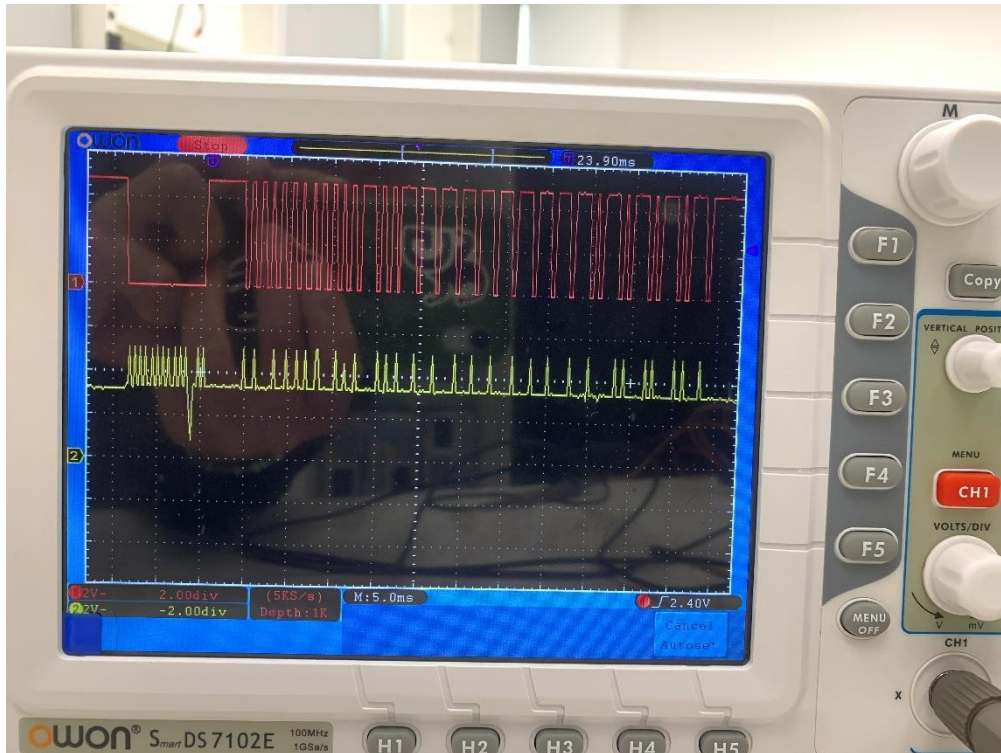


Figura: Osciloscopio — Trama NEC completa en pin RX del KY-022

6.3 Pulso de inicio NEC (AGC Burst)

El pulso de inicio es el marcador temporal que permite al receptor sincronizarse. Se compone de 9 ms de burst seguido de 4,5 ms de espacio. Se observa como es el pulso de mayor duración al inicio de la trama.

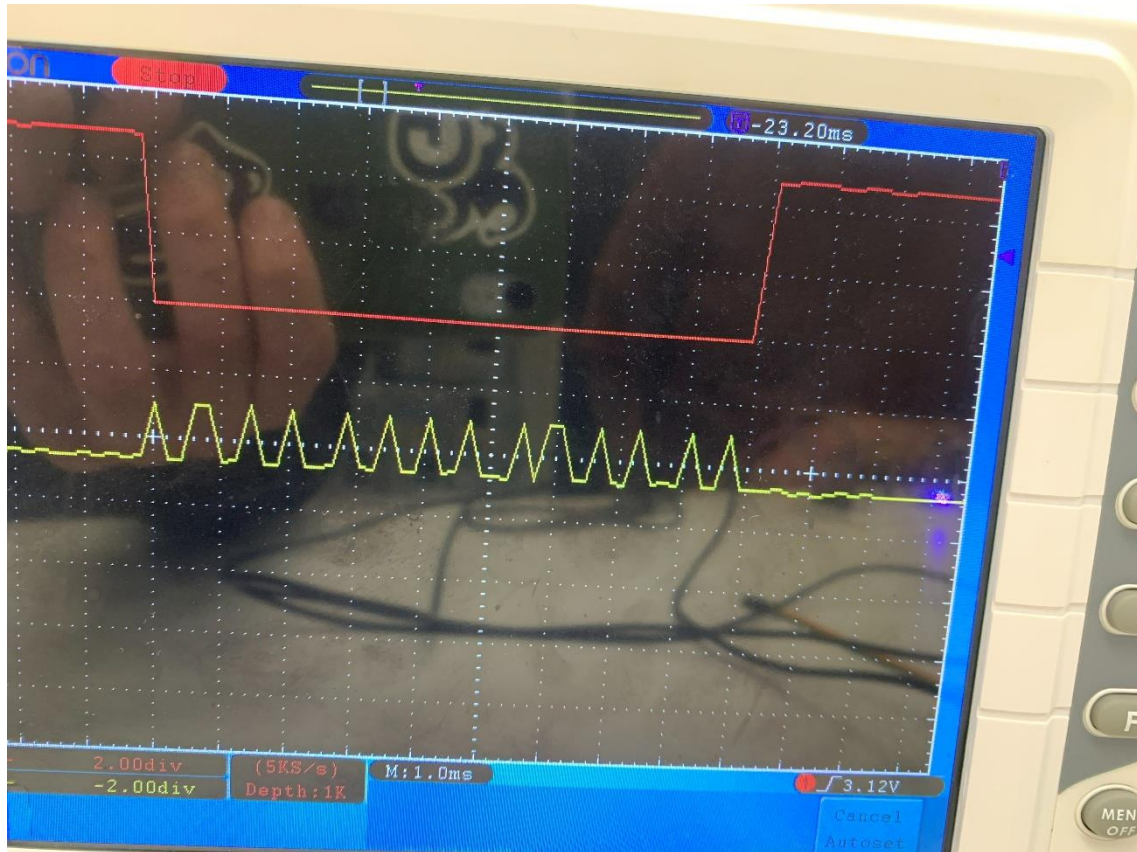


Figura: Osciloscopio — Pulso de inicio NEC: 9 ms burst + 4,5 ms espacio

6.4 Codificación bit '1' y bit '0' (Pulse Distance Modulation)

Se puede observar la diferencia de duración del espacio inter-pulso que distingue un '1' lógico (~2,25 ms total) de un '0' lógico (~1,12 ms total). La duración del burst es idéntica en ambos casos (~562,5 μ s).



Figura: Osciloscopio — Codificación PDM: bit '1' ($562\mu\text{s} + 1687\mu\text{s}$) vs bit '0' ($562\mu\text{s} + 562\mu\text{s}$)

6.5 Trama de comando completa [0xAA|R|G|B] — Maestro

Se observa la trama de 32 bits emitida por el Maestro. Los primeros 8 bits corresponden a la cabecera 0xAA (10101010b): se puede identificar la alternancia característica de bits '1' y '0' que le da a esta cabecera su firma espectral distintiva.

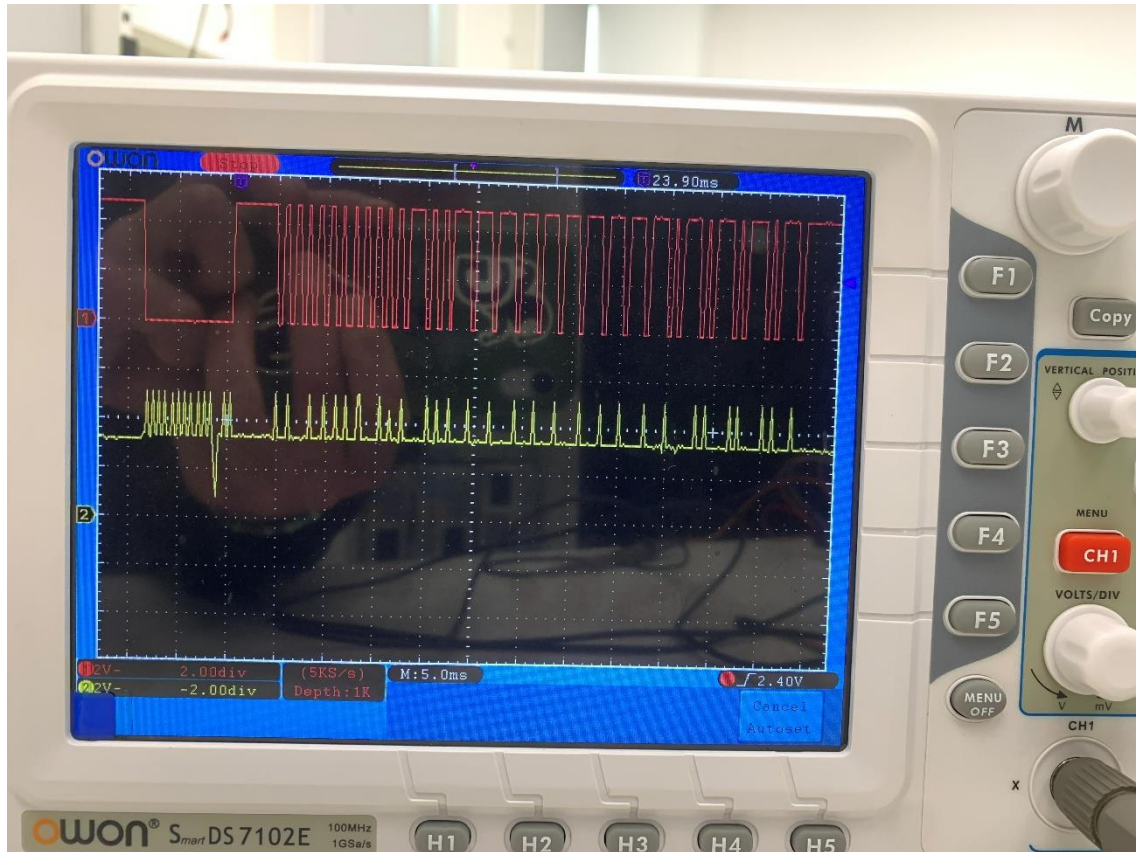


Figura: Osciloscopio — Trama CMD del Maestro: cabecera 0xAA + R=0 + G=255 + B=0 (VERDE)

7. Resultados y Conclusiones

7.1 Resultados obtenidos

El sistema implementado cumple satisfactoriamente con todos los objetivos del trabajo práctico:

- **Sincronización RGB bidireccional por IR:** ambos LEDs exhiben el mismo color tras cada pulsación exitosa, sin requerir ningún cable de interconexión entre los nodos.
- **Control de errores por cabecera:** tramas con cabecera inválida son descartadas por el Esclavo; el Maestro descarta ACKs con cabecera diferente de 0xBB o con valores RGB incorrectos.
- **ARQ funcional:** al obstruir el canal IR durante la transmisión del ACK, el Maestro reintenta y recupera la sincronización en el siguiente intento.
- **Persistencia:** al cortar y restaurar la alimentación de cualquier dispositivo, el sistema recupera el estado correcto de manera autónoma.
- **Alcance:** el sistema opera correctamente en el rango de 1 a 2 metros de distancia con visión directa entre módulos IR.
- **Análisis de señales por osciloscopio:** la capa física (portadora de 38 kHz) y la capa de enlace (trama NEC con pulso de inicio, bits codificados por PDM y secuencia de handshaking CMD+ACK) fueron validadas experimentalmente.

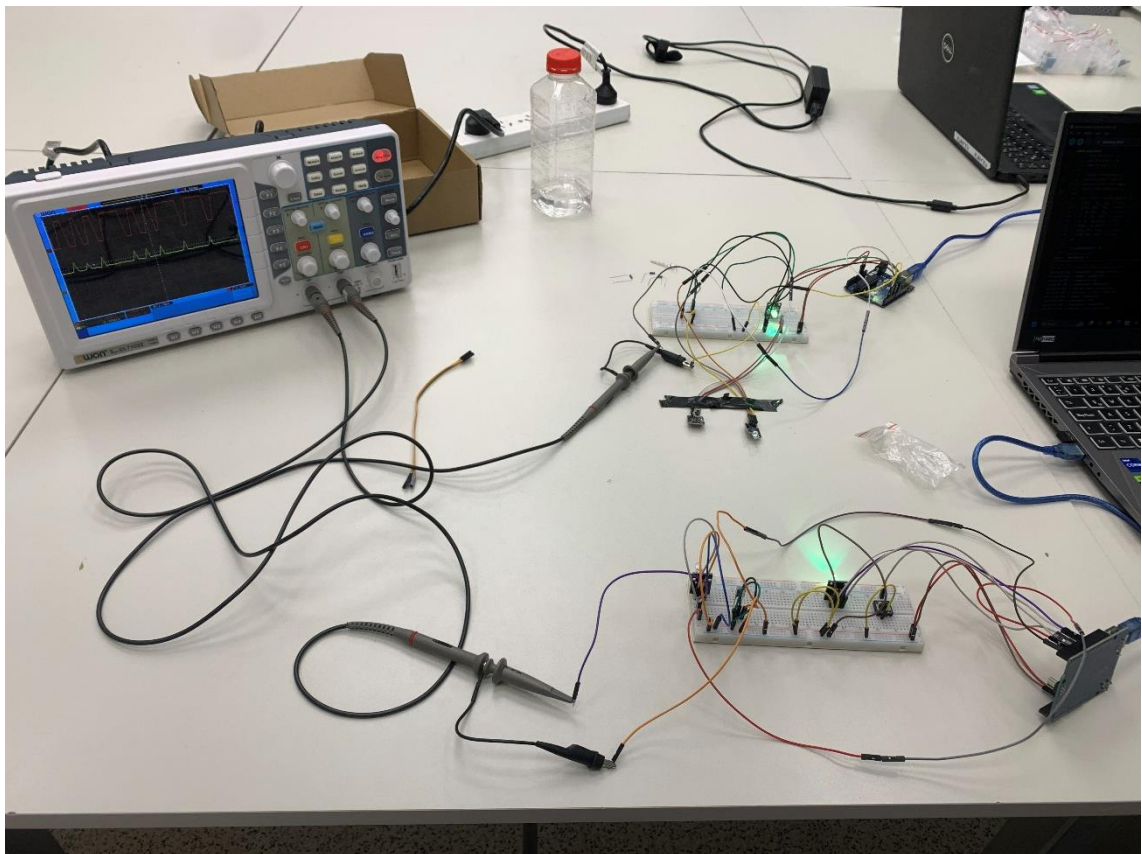


Figura: Sistema en funcionamiento — Ambos LEDs sincronizados en un color de la secuencia

7.2 Conclusiones

Este trabajo permitió abordar de manera práctica e integral los conceptos fundamentales de la comunicación de datos en sistemas embebidos: protocolización, codificación de canal, control de errores, gestión de flujo, handshaking bidireccional y persistencia de estado.

La transición del canal de ACK desde una línea digital cableada hacia un enlace infrarrojo pleno fue el paso técnico más significativo del proyecto. Esta evolución no solo eliminó la dependencia física entre los nodos, sino que enriqueció cualitativamente el ACK: de una señal binaria sin contenido semántico a una trama de 32 bits que permite al Maestro verificar la integridad de la operación byte a byte. Esta es exactamente la diferencia entre un sistema de señalización y un sistema de comunicación con control de errores real.

La elección de `sendNECRaw()` sobre `sendNEC()` fue una decisión técnica crítica derivada del profundo estudio de la API de la librería: `sendNEC()` está diseñada para el protocolo NEC original con dirección de 8 bits y comando de 8 bits, y trunca cualquier valor de 32 bits pasado como comando. `sendNECRaw()` permite usar el frame NEC como transporte de carga útil arbitraria de 32 bits, que es exactamente el requerimiento de un sistema que necesita transportar tres bytes de datos más un byte de identificación.

La asignación de pines requirió una comprensión detallada de la arquitectura de timers del ATmega328P. La diferencia entre los pines del Maestro (TX en D4) y del Esclavo (TX en D3) no es arbitraria: responde a la necesidad de evitar colisiones entre el Timer2 (usado por IRremote para la portadora de 38 kHz), los timers usados para PWM del LED y el timer del sistema (Timer0, usado por `millis()`).

7.3 Propuestas de mejora y escalabilidad

- **Reemplazo de magic byte por CRC-8:** el byte de cabecera (0xAA / 0xBB) actúa como primer filtro de validez de trama, pero no detecta corrupciones dentro de los bytes R, G o B. Agregar un byte de CRC-8 (Cyclic Redundancy Check de 8 bits) sobre los tres canales RGB aumentaría la robustez ante errores de bit, detectando todos los errores de 1 bit y las ráfagas de hasta 8 bits.
- **Número de secuencia para ARQ mejorado:** el ARQ actual es stop-and-wait con reintentos simples. Agregar un campo de número de secuencia de 4 bits permitiría distinguir ACKs de diferentes tramas y detectar duplicados sin depender de la idempotencia implícita.
- **Sincronización al arranque:** implementar un SYNC_REQUEST que el Maestro envíe al energizarse para consultar el estado del Esclavo antes de procesar el primer botón, garantizando coherencia desde el arranque cuando ambos dispositivos tienen historiales de EEPROM diferentes.
- **Escalado a múltiples esclavos:** agregar un campo de dirección de 4 bits en la trama permitiría un Maestro controlando hasta 15 esclavos independientes, sentando las bases de una red IR en estrella.
- **Analogía con protocolos de mayor escala:** los principios implementados (trama con cabecera, ACK con contenido, ARQ, persistencia) son directamente análogos a los mecanismos de Zigbee (IEEE 802.15.4), Bluetooth Low Energy y Z-Wave, donde la integridad del dato y la confirmación de recepción son los pilares de la confiabilidad de la red.

8. Referencias Bibliográficas

Tanenbaum, A. S., & Wetherall, D. (2011). Redes de computadoras (5.a ed.). Pearson Education.

Williams, A. (s.f.). NEC infrared transmission protocol. SB-Projects. <https://sbprojects.net/knowledge/ir/nec.php>

Joachimsmeier, A. (2024). IRremote Arduino library (v4.x) [Software]. GitHub. <https://github.com/Arduino-IRremote/Arduino-IRremote>

Arduino LLC. (2024). EEPROM library reference. <https://docs.arduino.cc/learn/built-in-libraries/eeprom/>

Vishay Semiconductors. (2023). TSOP382xx: IR receiver modules for remote control systems [Datasheet]. <https://www.vishay.com/docs/82491/tsop382.pdf>

Microchip Technology. (2018). ATmega328P 8-bit AVR microcontroller with 32K bytes in-system programmable flash [Datasheet]. <https://ww1.microchip.com/downloads/en/DeviceDoc/ATmega48A-PA-88A-PA-168A-PA-328-P-DS-DS40002061B.pdf>

Koopman, P. (2002). 32-bit cyclic redundancy codes for Internet applications. Proceedings of the International Conference on Dependable Systems and Networks (DSN), 459–468. <https://doi.org/10.1109/DSN.2002.1028931>

9. Repositorio y Material Multimedia

9.1 Código fuente — GitHub

El código fuente completo de ambos dispositivos, junto a los diagramas y este informe, se encuentran disponibles en el repositorio público de GitHub:

Repositorio: <https://github.com/bevilacquafrancisco/Comunicacion-de-Datos>

Estructura del repositorio:

Archivo / Carpeta	Descripción
Esclavo.ino	Firmware del Dispositivo 1 — Esclavo IR bidireccional
Maestro.ino	Firmware del Dispositivo 2 — Maestro IR bidireccional
Bevilacqua_Clement_CdD_T2_2026.pdf	Informe técnico final (este documento)
README.md	Descripción del proyecto, instrucciones de uso y dependencias

9.2 Imágenes y Videos de funcionamiento

Los videos demostrativos del sistema en funcionamiento se encuentran disponibles en el siguiente enlace:

https://drive.google.com/drive/folders/1P5FxIJzY4hZtIR2Mls0UCtlhy-UBNy5z?usp=drive_link